

2. Code Question 2

Amazon uses small, Roomba-shaped robots, called "Drives". They deliver large stacks of products to human workers by following set paths around the warehouse.

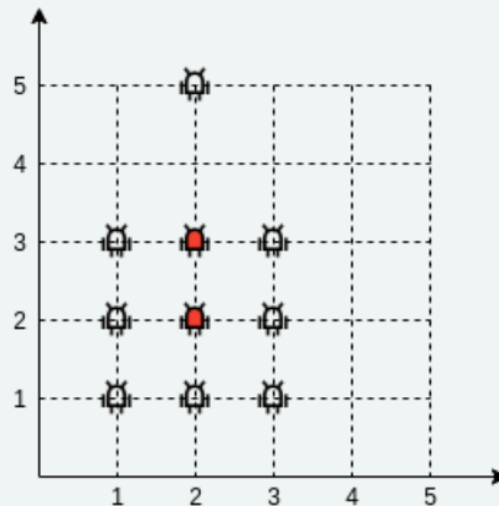
The warehouse can be represented in the form of a cartesian plane, where robots are located at integral points of the form (x, y) . When a product is to be delivered to some point (i, j) , the nearest robot is sought and chosen. Thus if a robot is surrounded by other robots nearby, it will seldom be chosen for work. More formally, a robot is said to be *idle* if it has a robot located above, below, left, and right of it. It is guaranteed that no two robots are at the same location.

Given the locations of n robots, find the number of *idle* robots.

Example

The x and y coordinates are given as two arrays, with the coordinates aligned by index: $x = [1, 1, 1, 2, 2, 2, 2, 3, 3, 3]$ and $y = [1, 2, 3, 1, 2, 3, 5, 1, 2, 3]$.

The 10 robots are arranged as follows:



- The robot at $(2, 2)$ is *idle* because it has robots at $(1, 2)$, $(3, 2)$, $(2, 3)$, and $(2, 1)$ directly to the left, right, up, and down respectively.
- The robot at $(2, 3)$ is *idle* because it has robots at $(1, 3)$, $(3, 3)$, $(2, 5)$, and $(2, 2)$ directly to the left, right, up, and down respectively.

There are 2 *idle* robots in this arrangement.

Function Description

Complete the function `getNumberIdle` in the editor below.

`getNumberIdle` has the following parameters:

`int x[n]`: $x[i]$ is the x -coordinate of the i^{th} robot, where $0 \leq i < n$.

`int y[n]`: $y[i]$ is the y -coordinate of the i^{th} robot, where $0 \leq i < n$.

Returns

`int`: the number of *idle* robots

Constraints

- $1 \leq n \leq 10^5$
- $-10^9 \leq x[i], y[i] \leq 10^9$

▼ Input Format For Custom Testing

The first line contains an integer, n , denoting the number of elements in array x .

Each line i of the next n subsequent lines (where $0 \leq i < n$) contains an integer x_i describing the i^{th} element of array x .

Next line contains an integer, n , denoting the number of elements in array y .

Each line i of the next n subsequent lines (where $0 \leq i < n$) contains an integer y_i describing the i^{th} element of array y .

▼ Sample Case 0

Sample Input For Custom Testing

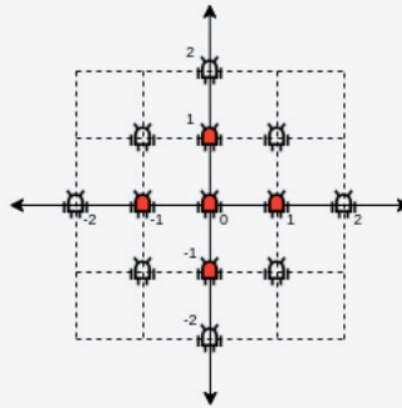
STDIN	FUNCTION
-----	-----
13	→ x[] size n = 13
0	→ x = [0, 0, 0, 0, 0, 1, 1, 1, 2, -1, -1, -2, -1]
0	
0	
0	
0	
1	
1	
1	
2	
-1	
-1	
-2	
-1	
13	→ y[] size n = 13
-1	→ y = [-1, 0, 1, 2, -2, 0, 1, -1, 0, 1, -1, 0, 0]
0	
1	
2	
-2	
0	
1	
-1	
0	
1	
-1	
0	
0	

Sample Output

5

Explanation

The cartesian plane with *idle* robots marked red can be represented as:



The robots at locations $(0, 0)$, $(1, 0)$, $(-1, 0)$, $(0, 1)$ and $(0, -1)$ are *idle* as they are surrounded by robots on all 4 sides.

▼ Sample Case 1

Sample Input For Custom Testing

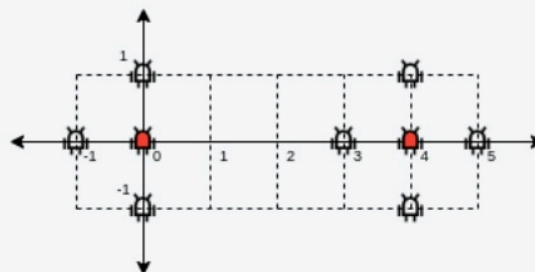
STDIN		FUNCTION
-----		-----
9	→	x[] size n = 9
0	→	x = [0, -1, 0, 0, 3, 5, 4, 4, 4]
-1		
0		
0		
3		
5		
4		
4		
4		
9	→	y[] size n = 9
0	→	y = [0, 0, 1, -1, 0, 0, 1, -1, 0]
0		
1		
-1		
0		
0		
1		
-1		
0		

Sample Output

2

Explanation

The cartesian plane with *idle* robots marked red can be represented as:



The robots at locations $(0, 0)$ and $(4, 0)$ are *idle* as they are surrounded by robots on all 4 sides.

```

1 > #!/bin/python3 ...
10 from collections import defaultdict
11 from bisect import bisect_left, bisect_right
12 #
13 # Complete the 'getNumberIdle' function below.
14 #
15 # The function is expected to return an INTEGER.
16 # The function accepts following parameters:
17 # 1. INTEGER_ARRAY x
18 # 2. INTEGER_ARRAY y
19 #
20
21 def getNumberIdle(x, y):
22     # Write your code here
23     x_map = defaultdict(list)
24     y_map = defaultdict(list)
25
26     for i in range(len(x)):
27         x_map[x[i]].append(y[i])
28         y_map[y[i]].append(x[i])
29
30     for key in x_map:
31         x_map[key].sort()
32
33     for key in y_map:
34         y_map[key].sort()
35
36     idle_count = 0
37
38     for i in range(len(x)):
39         current_x, current_y = x[i], y[i]
40
41         has_above = bisect_right(x_map[current_x], current_y) < len(x_map[current_x])
42         has_below = bisect_left(x_map[current_x], current_y) > 0
43         has_right = bisect_right(y_map[current_y], current_x) < len(y_map[current_y])
44         has_left = bisect_left(y_map[current_y], current_x) > 0
45
46         if has_above and has_below and has_left and has_right:
47             idle_count += 1
48
49     return idle_count
50
51

```

```
52 ✓ if __name__ == '__main__':  
53     fptr = open(os.environ['OUTPUT_PATH'], 'w')  
54  
55     x_count = int(input().strip())  
56  
57     x = []  
58  
59     for _ in range(x_count):  
60         x_item = int(input().strip())  
61         x.append(x_item)  
62  
63     y_count = int(input().strip())  
64  
65     y = []  
66  
67     for _ in range(y_count):  
68         y_item = int(input().strip())  
69         y.append(y_item)  
70  
71     result = getNumberIdle(x, y)  
72  
73     fptr.write(str(result) + '\n')  
74  
75     fptr.close()  
76
```